

WIPRO NGA Program _26SUB0726-JAVA J2EE INTEGRATION

Capstone Project Presentation – 23rd June 2026

Project Title –Asset Management System Presented

by - Varshitha G, Ramesh N

Asset Management System

Introduction

- The Asset Management System is a web-based application developed to automate the management of organizational assets throughout their lifecycle.
- The Asset Management System is a web-based application developed to automate the management of organizational assets throughout their lifecycle.
- The application eliminates manual asset tracking, improves asset visibility, reduces management overhead, and ensures efficient utilization of organizational resources.

Existing System

In traditional asset management systems:

- Asset records are maintained manually using spreadsheets.
- Asset allocation and return processes are time-consuming.
- Maintenance requests are tracked manually.
- Asset availability is difficult to monitor.
- Data inconsistency and duplication may occur.

Reporting and tracking require significant manual effort.,

These limitations reduce operational efficiency and increase the risk of asset mismanagement.

Proposed System

The proposed Asset Management System automates the complete asset lifecycle.

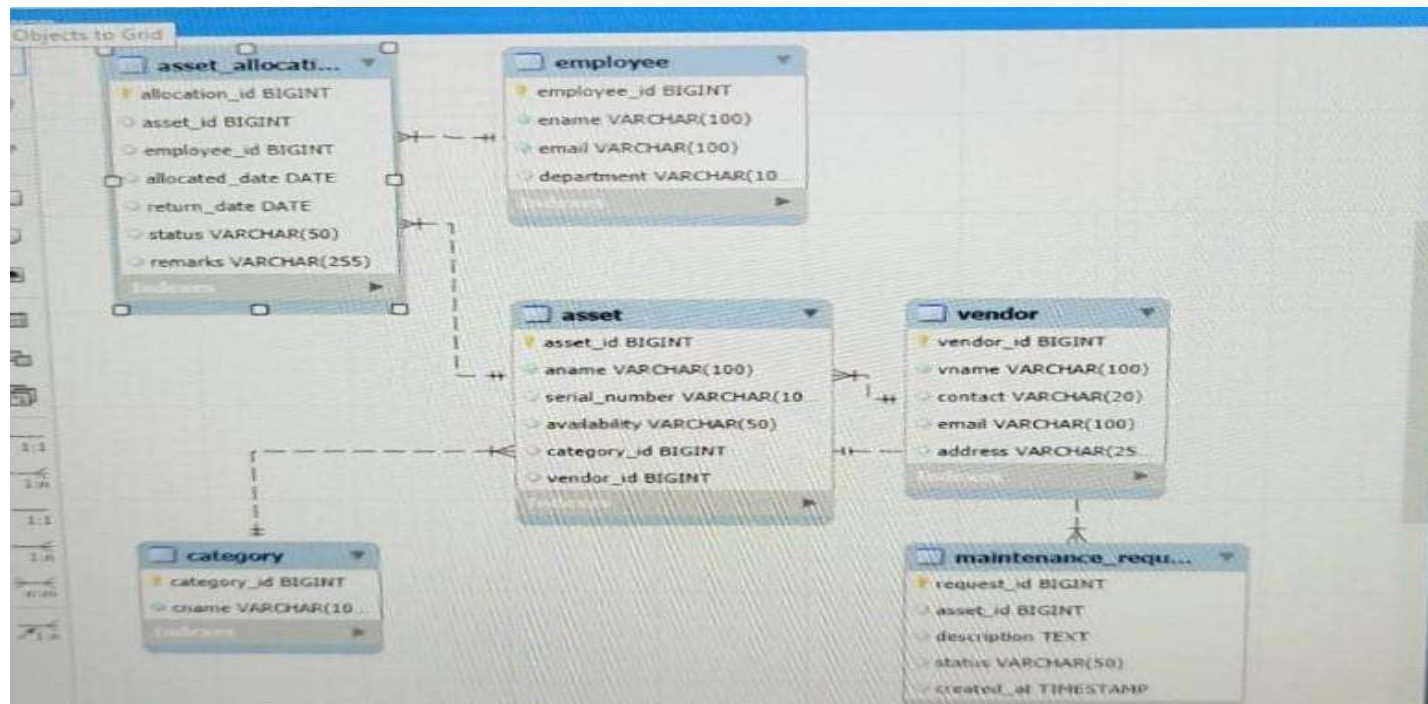
The system provides:

- Employee registration and authentication Asset
- registration and management
- Category and vendor management
- Asset allocation to employees
- Asset return processing
- Maintenance request management
- Asset tracking dashboard
- Role-based access control

Secure API access

Real time asset status monitoring

ER Diagram



Project Objectives

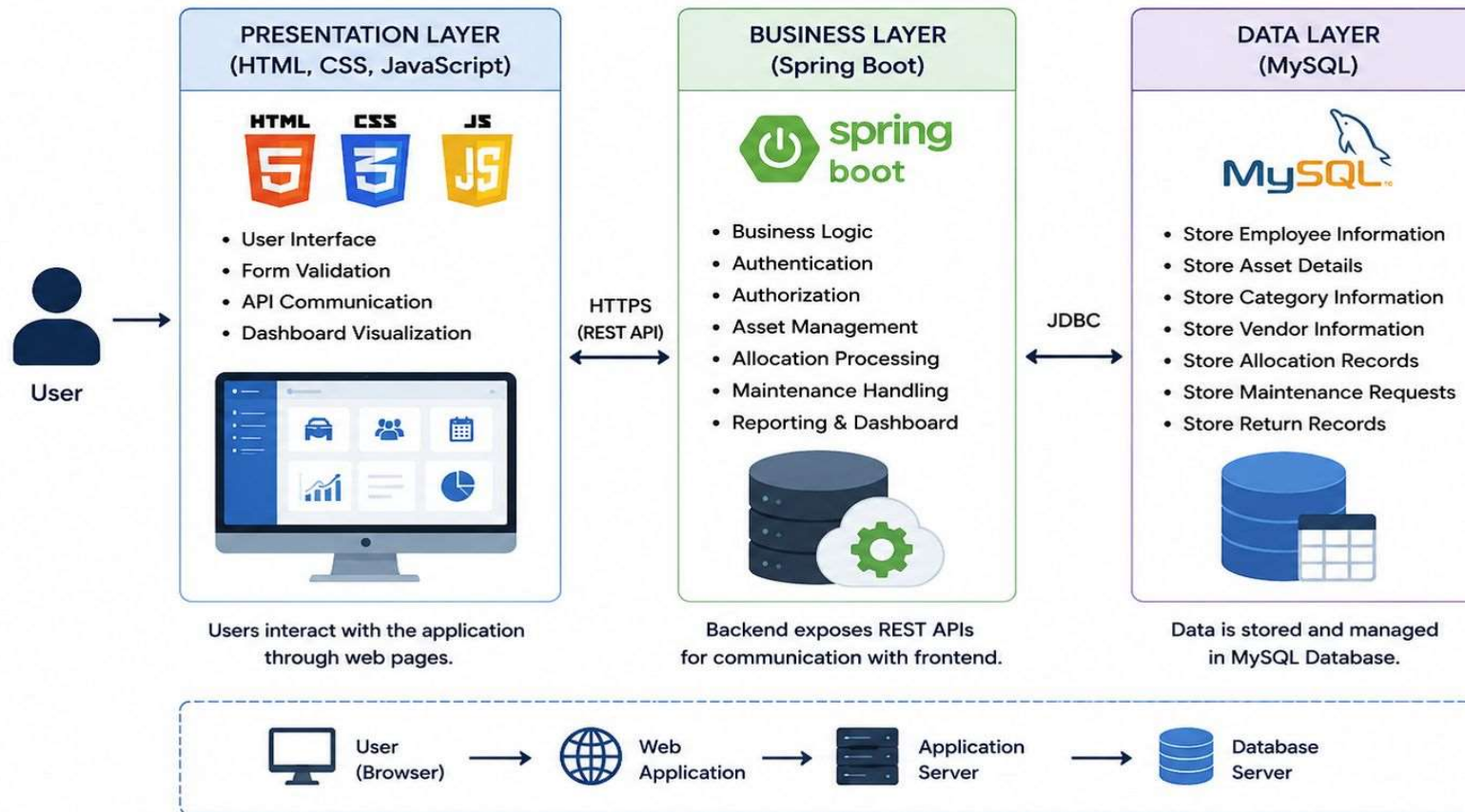
Objectives

- ❑ Automate asset registration and tracking
- ❑ Manage asset allocation efficiently
- ❑ Process asset returns
- ❑ Handle maintenance requests
- ❑ Maintain complete asset lifecycle records
- ❑ Provide secure authentication and authorization
- ❑ Deploy application using Docker and Kubernetes
- ❑ Automate CI/CD using Jenkins

System Architecture

THREE – TIER ARCHITECTURE

Asset Management System



Presentation Layer

The project follows a three-tier architecture.

Developed Using

- HTML
- CSS
- JavaScript
- Bootstrap

Responsibilities

- User Interface
- Form Validation
- API Communication
- Dashboard Display
- Asset Management Screens
- Users interact with the application through web pages designed for administrators and employees.

Business Layer

Developed Using

- Spring Boot

Responsibilities

- Asset Registration Logic
- Asset Allocation Processing
- Asset Return Processing
- Maintenance Request Handling
- Authentication & Authorization
- Dashboard Data Processing
- Validation and Exception Handling
- The backend exposes REST APIs for communication with the frontend.

Data Layer

Developed Using

MySQL Database

Responsibilities

- ❑ Store Employee Information
- ❑ Store Asset Details
- ❑ Store Category Information
- ❑ Store Vendor Information
- ❑ Store Asset Allocation Records
- ❑ Store Maintenance Requests
- ❑ Maintain Asset Lifecycle History

Data persistence is implemented using Spring Data JPA and Hibernate.

System Modules & Features

The Asset Management System consists of two primary user roles:

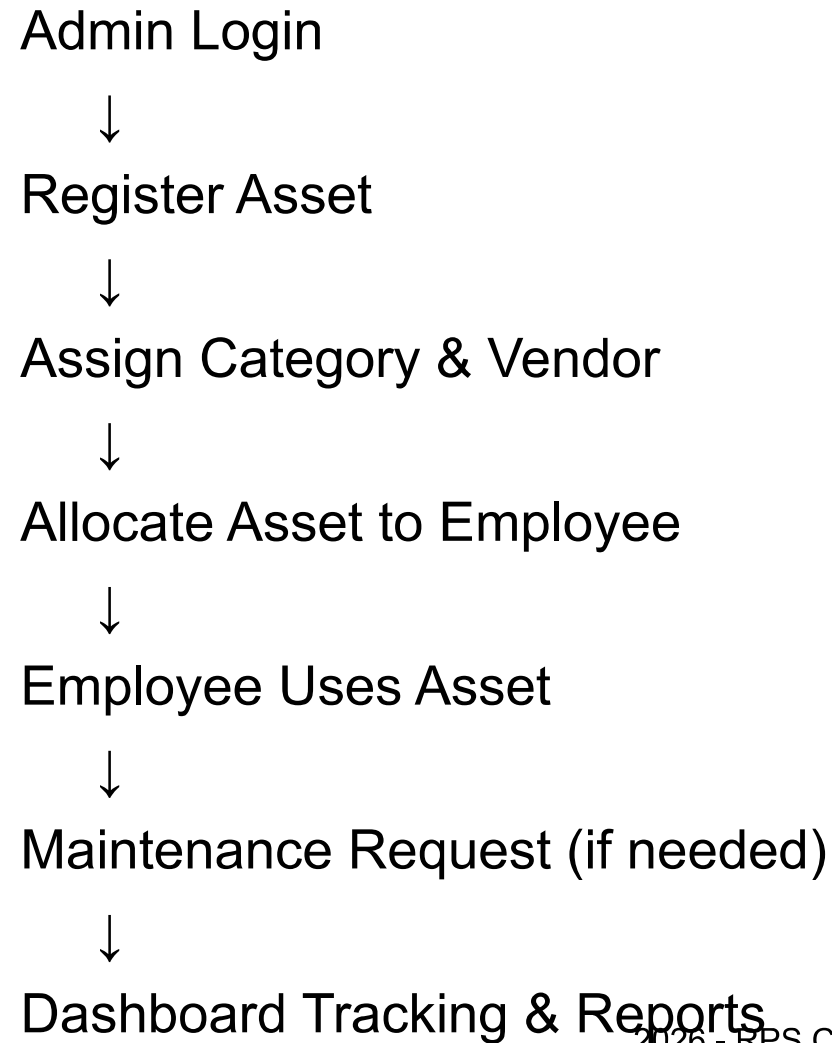
Administrator

- ❑ Manage Employees
- ❑ Manage Assets
- ❑ Manage Categories
- ❑ Manage Vendors
- ❑ Allocate Assets
- ❑ Process Asset Returns
- ❑ View Dashboard Reports
- ❑ Handle Maintenance Requests

Employee

- ❑ View Assigned Assets
- ❑ Request Asset Maintenance
- ❑ Track Asset Information
- ❑ Update Profile Information

System Workflow



DevOps, Deployment & Benefits

- The Asset Management System is developed using modern technologies including HTML,CSS Java Script for the frontend, Spring Boot for the backend, and MySQL for data storage. Security is implemented using Spring Security and , while Swagger is used for API documentation and Spring Boot Actuator for monitoring application health and metrics.
- To ensure portability and consistency across environments, the application is containerized using Docker. Kubernetes is used to orchestrate and manage the deployment of application containers, providing reliability and simplified deployment management. A Jenkins CI/CD pipeline automates the software delivery process by performing source code checkout, application build, testing, Docker image creation, and Kubernetes deployment.

Testing and Validation

Controller Layer Testing

- Employee APIs
- Asset APIs
- Vendor APIs
- Category APIs
- Maintenance APIs

Service Layer Testing

- Business Logic Validation
- Repository Mocking using Mockito

Benefits:

- ❑ Automated asset management
- ❑ Real-time asset tracking
- ❑ Secure authentication
- ❑ Efficient allocation and return process
- ❑ Maintenance request management
- ❑ Improved operational efficiency
- ❑ Docker & Kubernetes deployment Jenkins CI/CD automation
- ❑ Jenkins CI/CD automation

Drawbacks:

- ❑ Higher resource consumption
- ❑ Requires regular maintenance
- ❑ No RFID/IoT integration in current version
- ❑ Database remains a single point of failure
- ❑ Limited advanced reporting features

Conclusion

- The Asset Management System successfully automates the complete asset lifecycle including registration, allocation, return, maintenance, and tracking.
- The project improves organizational efficiency through centralized asset management, secure access control, automated workflows, and modern DevOps deployment practices using Docker, Kubernetes, and Jenkins.

THANK YOU